HelixOTA

HelixOTA

Universal, decoupled over-the-air updates — zero-brick by design.

Summary

Helix OTA is a universal, deeply decoupled over-the-air update system: a Go control plane plus per-OS client agents, designed to deliver safe, staged firmware/app updates to fleets ranging from a single board to millions of devices. Its first target is Android 15 on the Orange Pi 5 Max.

Short description

Helix OTA is a universal over-the-air update system — a Go control plane plus per-OS client agents — engineered for zero system corruption, validated uploads, and granular staged rollouts. Its first target is Android 15 on Orange Pi 5 Max, with Linux/Windows adapters planned.

Long description

Helix OTA is a universal, generic, deeply decoupled over-the-air (OTA) update system built for a single uncompromising promise: an update must never turn a working device into a brick. It comprises a Go server **control plane**, per-OS client **SDKs/ agents**, and a management **dashboard**, and it is engineered from the ground up to be embeddable into *any* operating system through pluggable OS adapters rather than rebuilt from scratch per platform. The first delivery target is Android 15 (all variants) on the Orange Pi 5 Max, where the build pipeline emits flashing images alongside a validated OTA `.zip` and mandatory hash files, so no artifact reaches a device without a verifiable fingerprint;

Linux, Windows, and other OSes sit on the roadmap behind the very same adapter seam, waiting only for their adapter — not a rewrite.

The design is organized around operator-stated hard guarantees treated as non-negotiable architectural invariants: zero system corruption, mandatory validation of every artifact before it is ever deployed, granular rollout (all-at-once or staged at 5/10/30…100% with halt-and-advance control), full observability of the fleet, and linear scale from a single board on a bench to millions of devices in the field. The locked architecture pairs device-side native Android A/B updates — AOSP `update_engine` with AVB/dm-verity and automatic boot-failure rollback — with a custom, decoupled Go control plane, so safety lives in the silicon-adjacent boot path *and* in the server, not in one fragile layer. Two seams are deliberately kept extractable: an OS-adapter seam that carries the promise of true universality, and a rollout-engine seam that makes staged campaigns OS-agnostic. The whole system is decomposed into six public, independently-versioned `ota-*` submodules — reusable building blocks rather than a monolith.

Helix OTA is currently in a specification/research and test-coverage build-out phase; the repository holds the authoritative design corpus, the documentation export pipeline, and submodule scaffolding, and it is explicit — per its anti-bluff governance — that a finished production server and agent do not yet exist. What ships today is the blueprint and its scaffolding, honestly labeled as such.

Why we built it

OTA is usually reinvented per device and per OS, and a bad update can brick a fleet. Helix OTA was built to be one universal, safety-first update system that any OS can adopt via adapters, with rollback and validation guarantees baked into the architecture rather than bolted on.

Why it’s a game-changer

It refuses to treat “never brick a device” and “roll out gradually and observably” as best-effort features you hope hold up under load — they are architectural invariants baked into the boot path and the control plane alike. And by making the rollout engine and OS layer swappable seams rather than hardwired assumptions, the same control plane can drive Android today and stand ready to drive other operating systems later purely by adding an adapter — no fork, no rewrite, no reinvention of the safety guarantees you already trust.

What’s innovative

- **Two extractable seams** — an OS-adapter seam and an OS-agnostic rollout engine — turning “universal” from a marketing word into a structural property of the codebase.
- **Defense-in-depth safety**: device-side native A/B (`update_engine`) + AVB/dm-verity + automatic boot-failure rollback, layered *on top of* server-side artifact validation — an update has to survive multiple independent gates before it can persist.
- **Catalogue-first, decoupled** decomposition into six reusable, independently-versioned `ota-*` submodules you can consume à la carte rather than swallowing a monolith.
- **HTTP/3 (QUIC) primary transport** with automatic HTTP/2 fallback and negotiated Brotli/gzip compression — modern, low-latency delivery that degrades gracefully instead of failing.
- **Anti-bluff engineering**: design and status are explicitly marked as spec-phase, and nothing unbuilt is ever claimed as shipped — honesty enforced as a first-class engineering value, not a disclaimer in the footnotes.

Biggest technical challenges & how we solved them

- **Guaranteeing a bad update never bricks a device** — the hardest promise in OTA. Solved by mandating device-side native Android A/B: `update_engine` writes to the inactive slot while the live slot keeps running, AVB/dm-verity cryptographically verifies the boot chain, and if the new slot fails to boot the device rolls back automatically — all backstopped by mandatory pre-deploy artifact validation so a corrupt payload is caught before it ever leaves the server.
- **One system, many operating systems** — solved by refusing to bake Android assumptions into the core. A pluggable OS-adapter seam isolates platform specifics, and an OS-agnostic rollout engine seam keeps campaign logic portable, each kept as a separate submodule so a new OS is an addition, never a surgery on the whole.
- **Staged, halt-able rollouts** — solved with a dedicated rollout engine that reasons in percentage cohorts with success/error thresholds and explicit halt/advance control, deliberately kept free of HTTP coupling so the same engine can drive campaigns independent of transport.

Tech stack

- **Go + Gin** — chosen for its concurrency model and lean deployment footprint; powers the control plane, the rollout

engine, and the artifact validators, exposing the REST `/api/v1` primary surface.
- **Kotlin/KMP** — chosen so the on-device Android OTA agent can share logic across targets; owns the full device loop of poll / download / verify / apply / report.
- **HTTP/3 (QUIC) → HTTP/2** — QUIC chosen as primary transport for low-latency, resilient delivery over lossy mobile links, with automatic HTTP/2 fallback so no device is stranded; **Brotli/gzip** negotiated per-request to shrink payloads.
- **PostgreSQL** — chosen for relational integrity across the device registry, campaigns, and telemetry, where correctness of fleet state matters more than raw write speed.
- **MinIO / S3** — chosen as the artifact blob store so large firmware images live in commodity object storage, decoupled from the relational layer.
- **AOSP `update_engine` + AVB/dm-verity + `boot_control`** — chosen because reusing Android's own battle-tested Virtual A/B and verified-boot machinery is safer than inventing a bespoke updater; used to drive slot swaps and cryptographic boot verification on-device.
- **React** — chosen for the management dashboard where operators log in, upload artifacts, drive rollouts, and watch fleet health in one place.
- **OpenTelemetry + Prometheus/Grafana** — chosen for vendor-neutral instrumentation; used to make every stage of a rollout observable in metrics and dashboards rather than guessed at.

Status & honesty notes

- **Status: in-development.** Per the project's own anti-bluff governance, there is **no working production server or agent yet** — this is a specification/research and test-coverage build-out phase. The repository holds the authoritative design corpus, the docs export pipeline, and submodule scaffolding.
- The six public reusable submodules (`ota-protocol`, `ota-artifact-validator`, `ota-rollout-engine`, `ota-update-engine-bridge`, `ota-android-agent`, `ota-telemetry-schema`) exist under `github.com/HelixDevelopment/`.
- Test-coverage and latency figures in the repository are the project's own in-progress ledger, not independently confirmed. HelixConstitution clause numbers cited in the README are UNVERIFIED.
- **License: Apache-2.0.**

Priority tier: Helix-primary.